

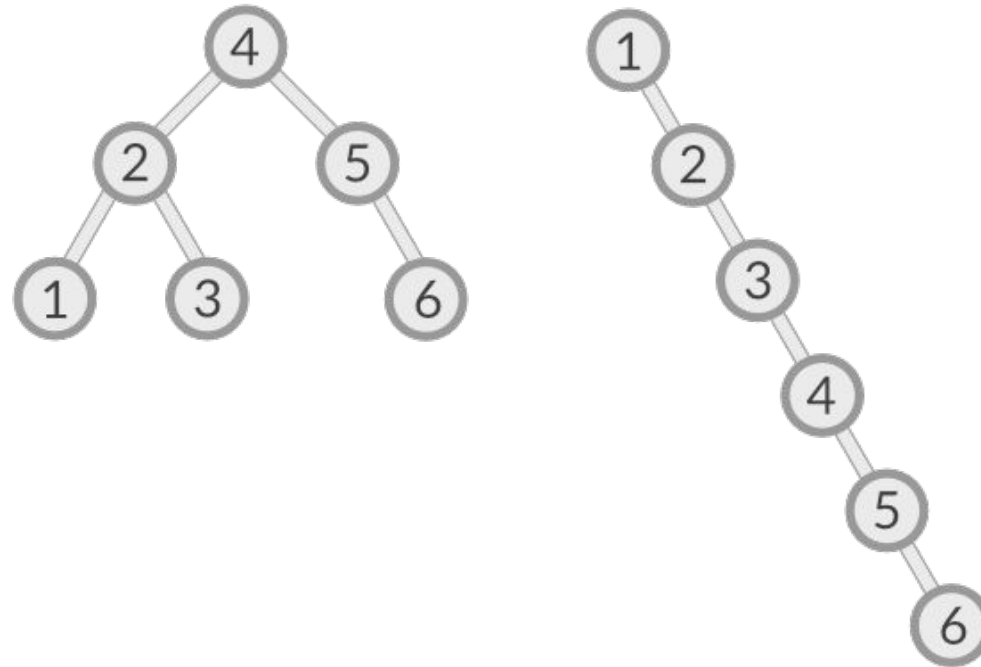
Introduction to Algorithms

Levon Prazyan

Course Overview

- Introduction
- Sorting and Searching
- Arrays and Lists
- Stacks and Queues, Double Ended Queue
- Heaps
- **Trees**
- Hash Tables
- Dynamic Programming
- Greedy Algorithms
- Graph Algorithms
- Network Flow
- NP-Completeness

Balanced vs Unbalanced



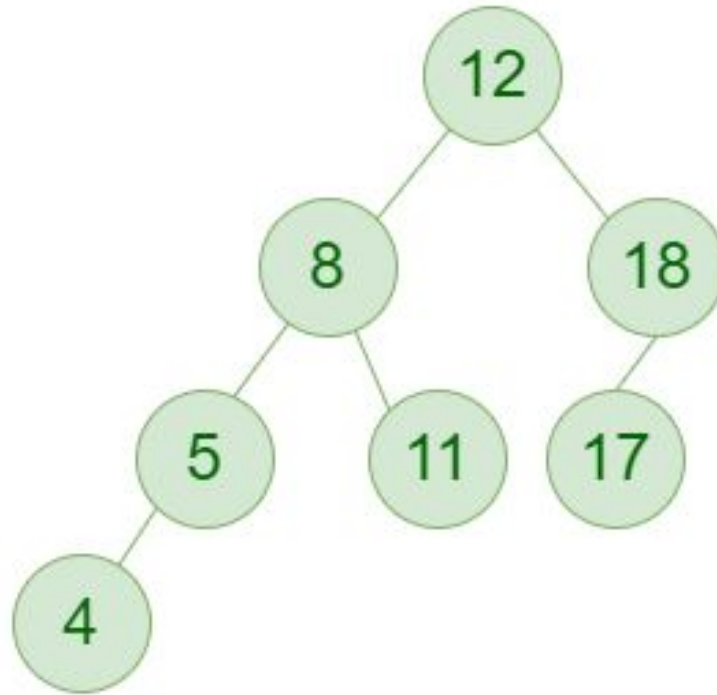
- Balanced BSTs guarantee that basic dynamic-set operations, such as *SEARCH*, *PREDECESSOR*, *SUCCESSOR*, *MINIMUM*, *MAXIMUM*, *INSERT*, and *DELETE* take $O(\log n)$ time in the worst case.

AVL Tree: Definition

An **AVL tree** is a binary search tree that is height balanced: for each node x , the heights of the left and right subtrees of x differ by at most 1.

The AVL tree implementation maintains an extra attribute in each node: $x.h$ is the height of node x .

AVL Tree: Example

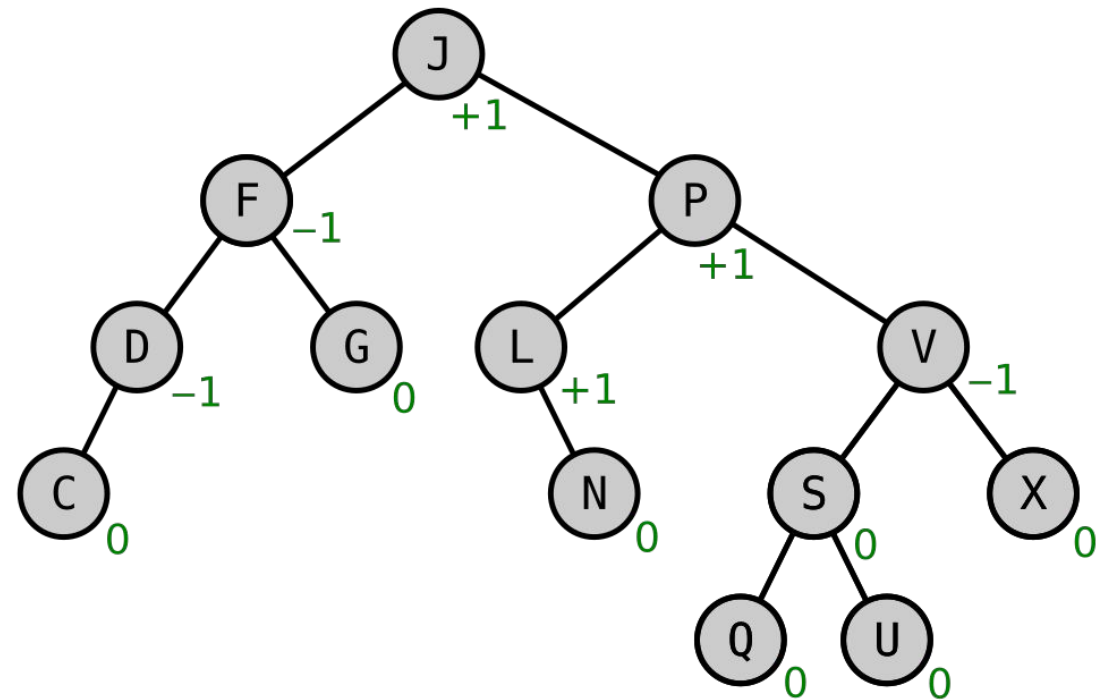


AVL Tree: Balance Factor

• *Balance Factor* = $height(right\ subtree) - height(left\ subtree)$

Valid values are -1, 0, 1

BF(X) < 0 is called "left-heavy"
BF(X) > 0 is called "right-heavy",
BF(X) = 0 is sometimes simply called "balanced".



AVL Tree: Height (1)

Lemma: An AVL tree with n nodes has height $O(\log n)$.

Proof:

Define: $N(h)$: minimum number of nodes in an AVL tree of height h .

To get the minimum number of nodes in a height- h AVL tree, we assume it's just balanced enough to be AVL-valid. That is:

- One subtree has height $h-1$
- The other has height $h-2$

This gives the recurrence:

$$N(h) = 1 + N(h - 1) + N(h - 2)$$

AVL Tree: Height (2)

$$N(h) = 1 + N(h - 1) + N(h - 2) ; N(0) = 1 ; N(1) = 2$$

Very Similar to Fibonacci numbers:

$$N(h) \geq F_h$$

$$F(k) = \phi^k ; \phi = \frac{1 + \sqrt{5}}{2} \approx 1.618$$

$$N(h) \geq \phi^h$$

$$n \geq N(h) \geq \phi^h \Rightarrow h \leq \log_{\phi} n$$

$$h = O(\log n)$$

AVL Tree: Height (3)

$$N(h) = 1 + N(h-1) + N(h-2) ; N(0) = 1 ; N(1) = 2$$

$$N(h) \geq 2N(h-2)$$

$$N(h) \geq 2N(h-2) \geq 2^2 N(h-4) \geq \dots \geq c2^{h/2}$$

$$n \geq N(h) \geq c2^{h/2}$$

$$\frac{h}{2} \leq \log n - \log c$$

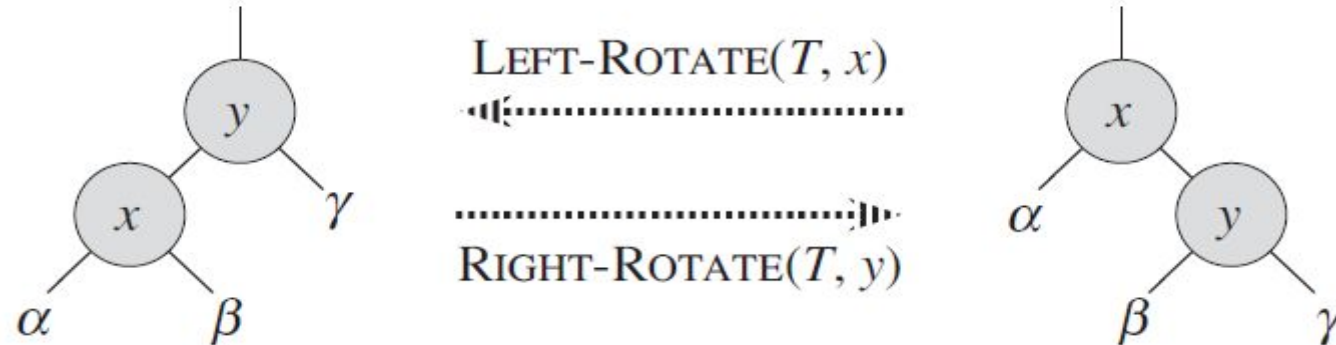
$$h = O(\log n)$$

Rotations: Left-Rotate And Right-Rotate

Because *TREE – INSERT* and *TREE – DELETE* modify the tree, the result may violate the AVL properties.

To restore these properties, we must change the pointer structure (rotate).

Rotation operation change the pointer structure that preserves the binary-search-tree invariants.



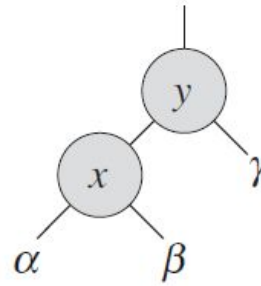
AVL Tree: Left Rotate

LEFT-ROTATE(T, x)

```

1   $y = x.right$ 
2   $x.right = y.left$ 
3  if  $y.left \neq T.nil$ 
4       $y.left.p = x$ 
5   $y.p = x.p$ 
6  if  $x.p == T.nil$ 
7       $T.root = y$ 
8  elseif  $x == x.p.left$ 
9       $x.p.left = y$ 
10 else  $x.p.right = y$ 
11  $y.left = x$ 
12  $x.p = y$ 
13  $x.height = 1 + \max(\text{height}(x.left),$ 
                        $\text{height}(x.right));$ 
14  $y.height = 1 + \max(\text{height}(y.left),$ 
                        $\text{height}(y.right));$ 

```

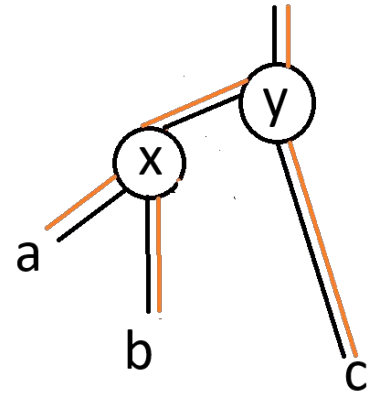
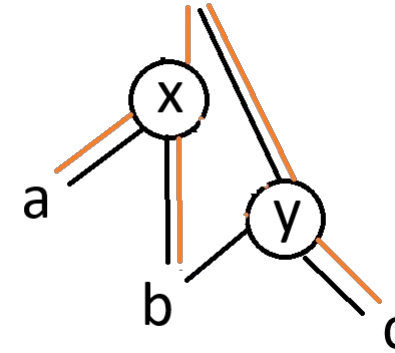
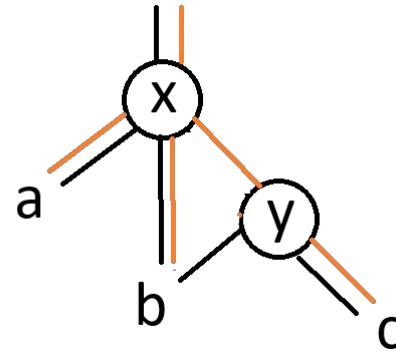
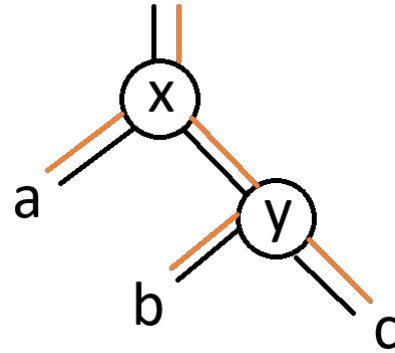
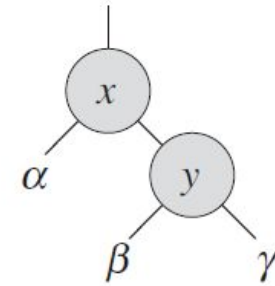


LEFT-ROTATE(T, x)

.....

.....

RIGHT-ROTATE(T, y)

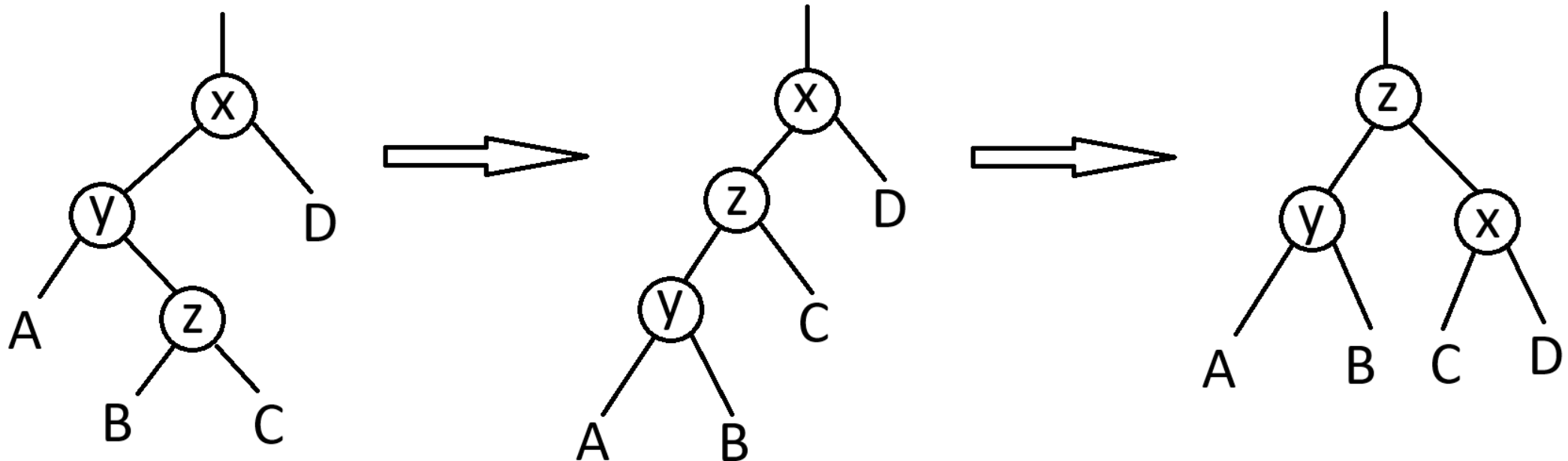


Rotations: Left-Right Rotate

LeftRightRotate(x):

LeftRotate(x.left)

return RightRotate(x)

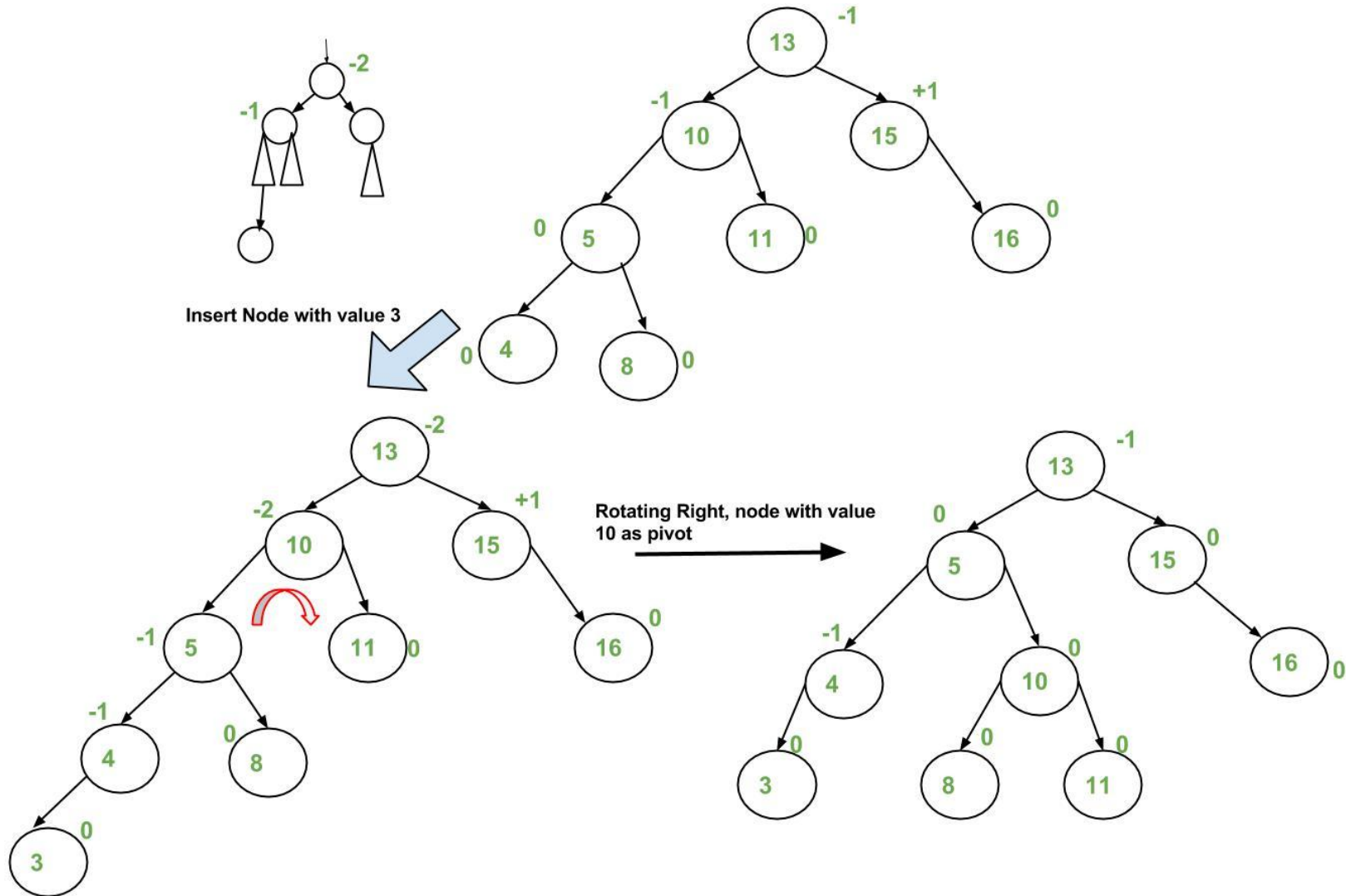


AVL Tree: Insert (1)

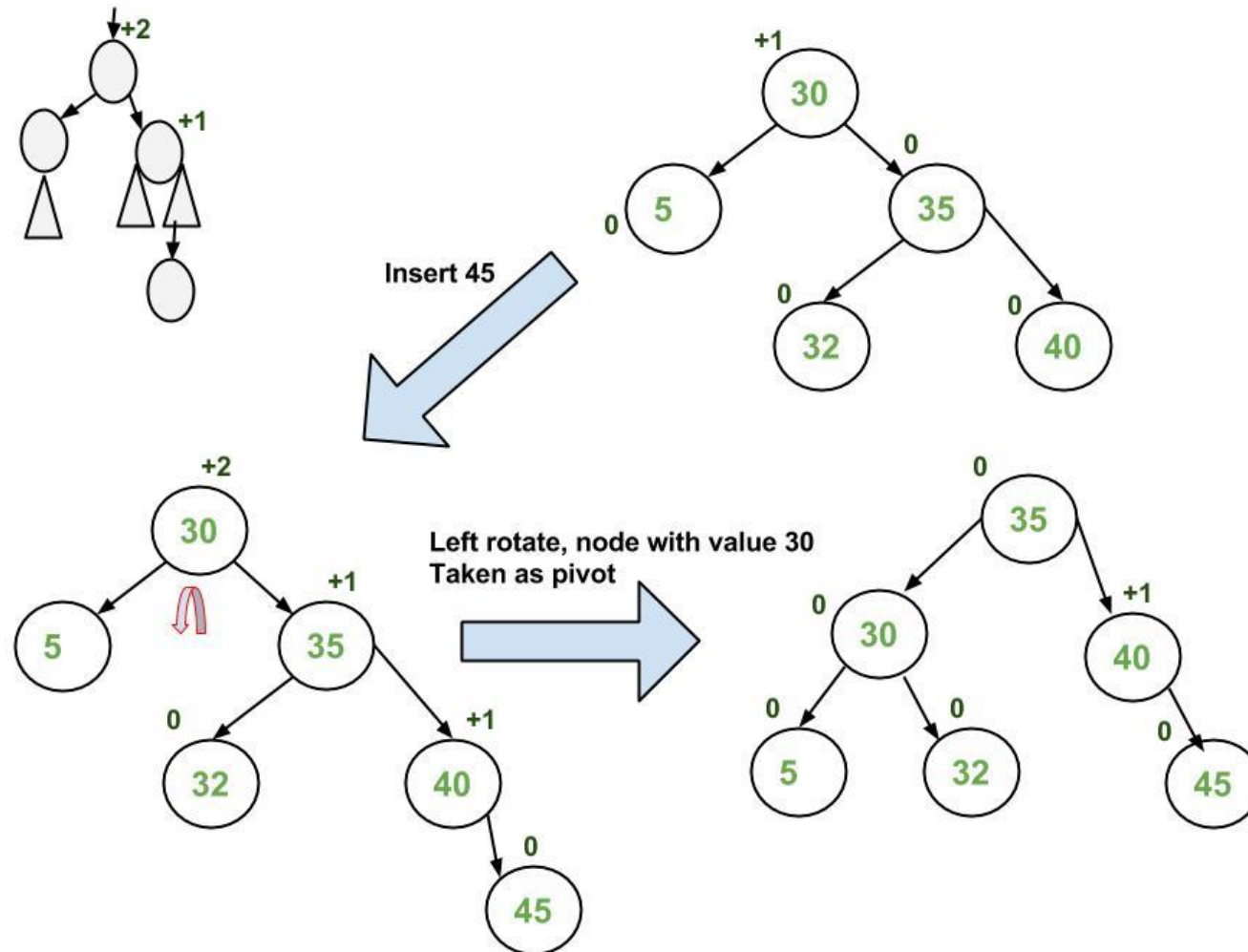
Let the newly inserted node be w

- Perform BST insert for w .
- Starting from w , travel up and find the first unbalanced node.
 - Let z be the first unbalanced node,
 - y be the child of z that comes on the path from w to z and
 - x be the grandchild of z that comes on the path from w to z .
- Re-balance the tree by performing appropriate rotations on the subtree rooted with z . There can be 4 possible cases that need to be handled as x , y and z can be arranged in 4 ways.
 - y is the left child of z and x is the left child of y (Left Left Case)
 - y is the left child of z and x is the right child of y (Left Right Case)
 - y is the right child of z and x is the right child of y (Right Right Case)
 - y is the right child of z and x is the left child of y (Right Left Case)

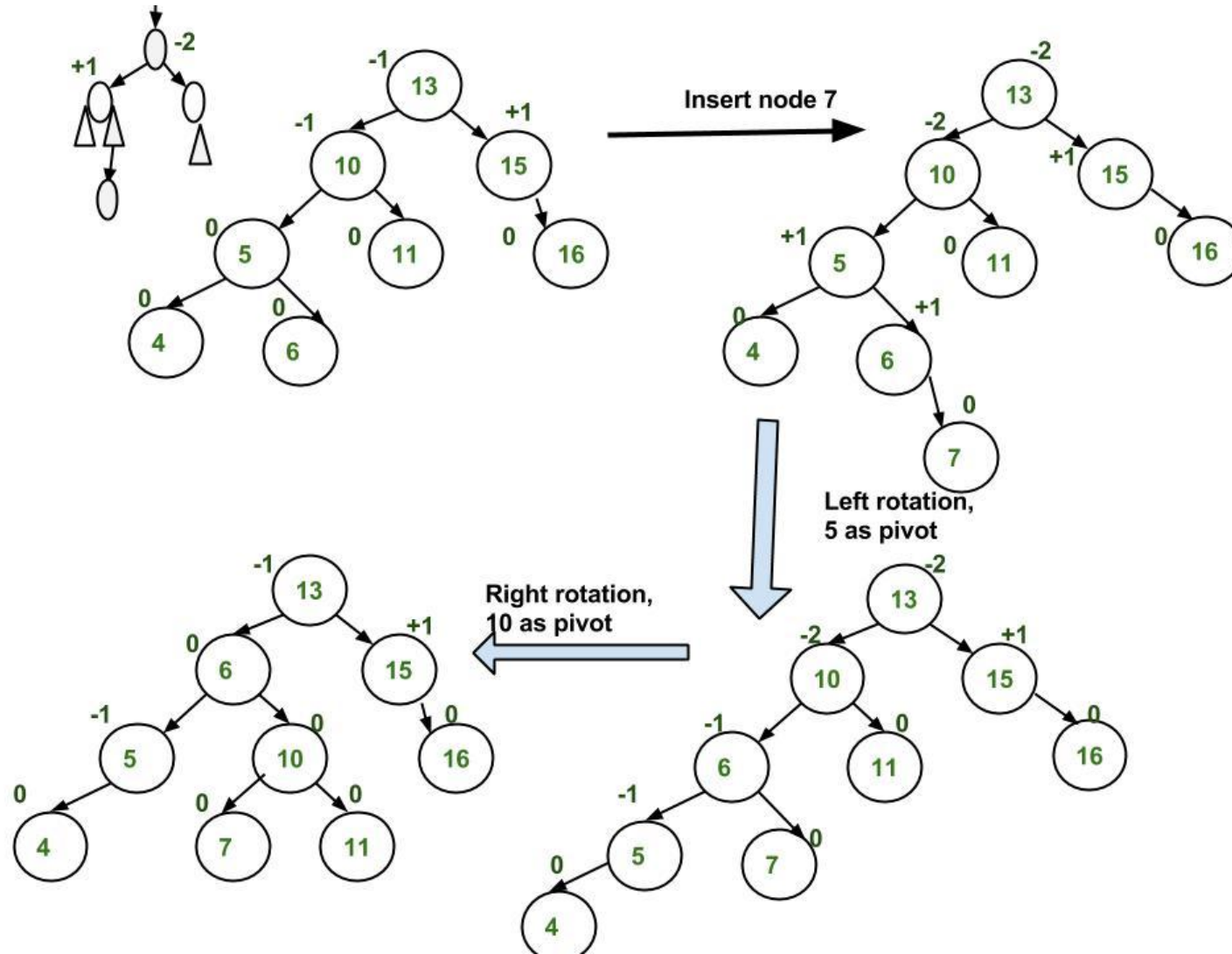
AVL Tree: Insert Example (1)



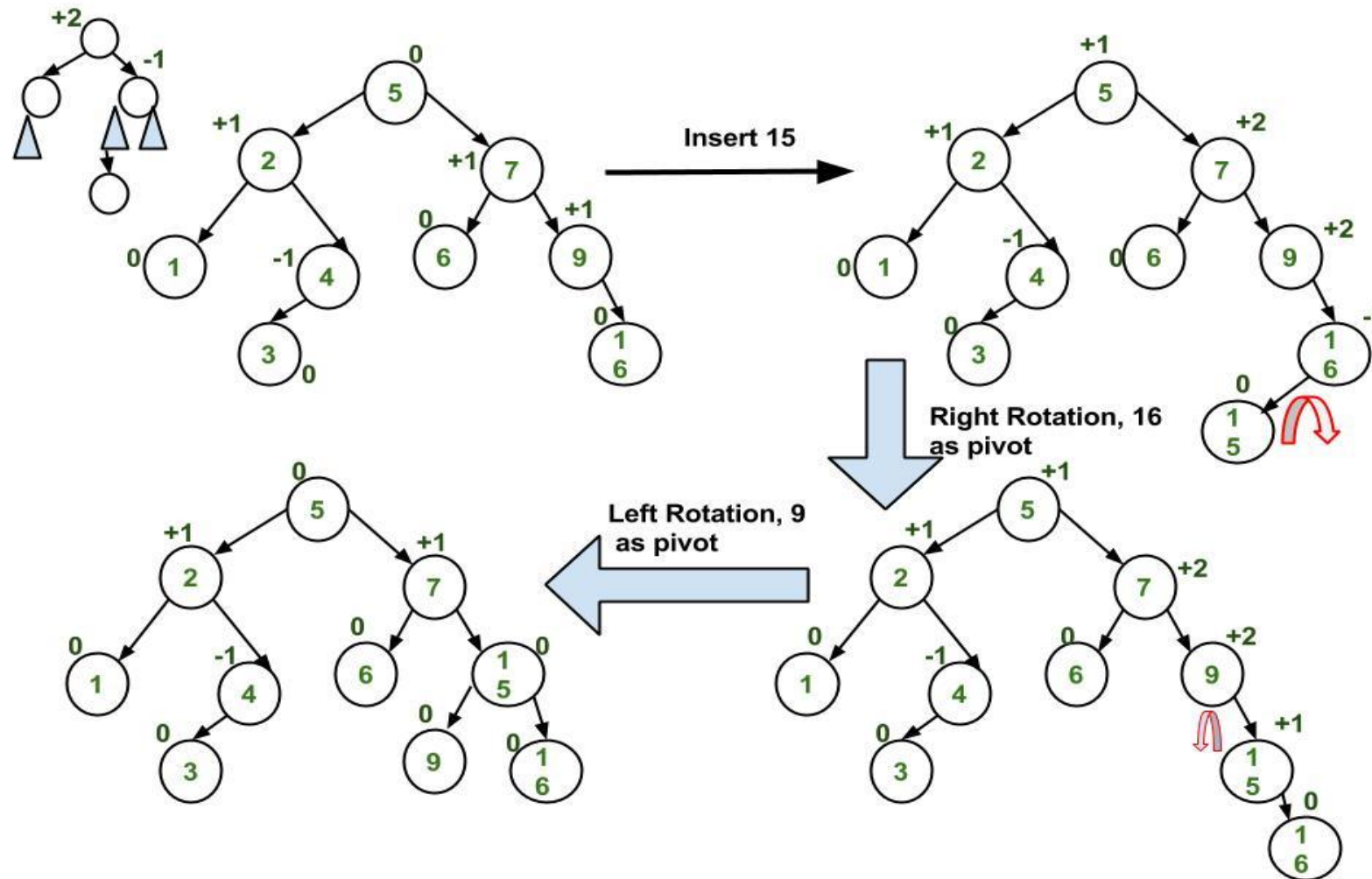
AVL Tree: Insert Example (2)



AVL Tree: Insert Example (3)



AVL Tree: Insert Example (4)



AVL Tree: Insert (2)

AVL-INSERT(T, z)

```
1   $y = T.nil$ 
2   $x = T.root$ 
3  while  $x \neq T.nil$ 
4       $y = x$ 
5      if  $z.key < x.key$ 
6           $x = x.left$ 
7      else  $x = x.right$ 
8   $z.p = y$ 
9  if  $y == T.nil$ 
10      $T.root = z$ 
11  elseif  $z.key < y.key$ 
12      $y.left = z$ 
13  else  $y.right = z$ 
14  AVL-INSERT-FIXUP( $T, z$ )
```

```
1  AVL-Insert-Fixup
2  while  $z \neq T.root$  and  $z.p \neq T.root$ 
3       $z.h = 1 + \max(z.left.h, z.right.h)$  //  $T.nil.h = -1$ 
4       $y = z.p;$        $y.h = 1 + \max(y.left.h, y.right.h)$ 
5       $x = y.p;$        $x.h = 1 + \max(x.left.h, x.right.h)$ 
6      if  $BF(x) > 1$  or  $BF(x) < -1$ 
7          if  $y = x.left$  and  $z = y.left$ 
8              Right_Rotate( $T, x$ )
9          if  $y = x.left$  and  $z = y.right$ 
10              Left_Right_Rotate( $T, x$ )
11          if  $y = x.right$  and  $z = y.right$ 
12              Left_Rotate( $T, x$ )
13          if  $y = x.right$  and  $z = y.left$ 
14              Right_Left_Rotate( $T, x$ )
15          break;
16   $z = z.p$ 
```

AVL Tree: Insert (3)

```
1 Insert(node, key):
2     if node is null:
3         return new Node(key)
4     if key < node.key:
5         node.left = Insert(node.left, key)
6     else if key > node.key:
7         node.right = Insert(node.right, key)
8     else: return node // Duplicate keys not allowed
9
10    node.height = 1 + max(height(node.left), height(node.right)) // Update ancestor height
11    balance = height(node.left) - height(node.right) // BF
12    if balance > 1 and key < node.left.key: // Case 1 - Left Left
13        return RightRotate(node)
14    if balance < -1 and key > node.right.key: // Case 2 - Right Right
15        return LeftRotate(node)
16    if balance > 1 and key > node.left.key: // Case 3 - Left Right
17        node.left = LeftRotate(node.left)
18        return RightRotate(node)
19    if balance < -1 and key < node.right.key: // Case 4 - Right Left
20        node.right = RightRotate(node.right)
21        return LeftRotate(node)
22    return node
```

AVL Tree: Insert (4)



AVL vs Red-Black

Feature	AVL Tree	Red-Black Tree
Balance Strictness	Stricter (balance factor: -1, 0, +1)	Looser (black height constraints)
Height Guarantee	Lower height $\Rightarrow$ closer to perfect balance	Higher height $\Rightarrow$ slightly less balanced
Insert/Delete Speed	Slower (more rotations may be needed)	Faster (fewer rotations)
Search Performance	Better (due to better balance)	Slightly worse
Rotations per Insert	Up to 2	Up to 2
Rotations per Delete	Up to $O(\log n)$	Often $O(1)$
Use Case Suitability	Good for read-heavy scenarios	Good for write-heavy scenarios
Implementation	More complex due to multiple rotation cases	Slightly simpler (uses color flips & fewer cases)

AVL Tree: Exercises

1. Insert the following keys into an empty AVL tree:
 - a) 10, 20, 30, 25, 28, 27, 5
 - b) 30, 20, 10, 14, 12, 13, 40
 - c) 40, 10, 30, 20, 15, 16, 18
 - d) 1, 2, 3, 4, 5, 10, 9, 8, 7, 6
2. Explain why at most two rotations are sufficient during insertion in an AVL tree.
3. Write an AVL delete.

AVL Tree: Delete

```
00 deleteNode(root, key):
01     if root == NULL:         return NULL
02     if key < root.key:
03         root.left = deleteNode(root.left, key)
04     else if key > root.key:
05         root.right = deleteNode(root.right, key)
06     else:
07         if root has no child:
08             return NULL
09         else if root has one child:
10             return root.left OR root.right
11         else:
12             temp = minValueNode(root.right)
13             root.key = temp.key
14             root.right = deleteNode(root.right, temp.key)
```

```
16     root.height = 1 + max(height(root.left), height(root.right))
17     balance = height(root.right) - height(root.left)
18
19     if balance < -1:
20         if height(root.left.left) > height(root.left.right):
21             return rightRotate(root)           // Left-Left Case
22         else:
23             root.left = leftRotate(root.left)
24             return rightRotate(root)           // Left-Right Case
25     if balance > 1:
26         if height(root.right.right) > height(root.right.left):
27             return leftRotate(root)           // Right-Right Case
28         else:
29             root.right = rightRotate(root.right)
30             return leftRotate(root)           // Right-Left Case
31     return root
```



Thank You